

Laboratório 04 Sincronização de Tarefas e Processos (parte 1)

Introdução

O objetivo deste laboratório é aplicar mecanismos de sincronização, tanto a processos como a tarefas, de forma a garantir a correta coordenação e execução concorrente. Como já referido, as threads (tarefas) podem ser vistas como unidades de execução mais leves do que os processos, sendo que muitos sistemas operativos suportam processos com múltiplas threads. As threads de um mesmo processo partilham o seu espaço de endereçamento, variáveis globais, descritores de ficheiros, sinais, entre outros recursos.

No entanto, cada thread possui informação própria, como o program counter, os registos de execução, o estado de execução e a sua stack.

As operações sobre threads, como criação, destruição e comutação de contexto, são geralmente mais rápidas do que as mesmas operações sobre processos. Isto deve-se ao facto das threads partilharem recursos do processo principal, o que não requer uma replicação do espaço de endereçamento e mudança de contexto.

[Ex.2b: pode auxiliar a resolução com recurso a ferramentas de IA].

Revisão

Comece por rever as atividades propostas nos slides 35 e 36 da aula 04.

Exercício 1. Casa da Sorte

Uma casa de lotarias fez uma emissão de N bilhetes para vender (N_TICKETS).

Cada cliente tenta comprar repetidamente bilhetes, até que os bilhetes esgotem (para aumentar as suas chances de ganhar).

Quando os bilhetes esgotam, os clientes devem, simplesmente, sair (terminar).

Depois de todos os clientes saírem, será anunciado/confirmado pela casa qual a quantidade de bilhetes vendidos (que, teoricamente, deverá bater certo com a quantidade de bilhetes inicialmente emitidos).

Para simular o cenário descrito, cada cliente será representado por uma thread e devem ser assegurados os seguintes requisitos:

- O número total de clientes existentes é igual a N_CLIENTS (cada um representado por uma thread);
- Cada cliente é uma thread que está infinitamente a tentar comprar mais bilhetes (a cada iteração com intervalos de 2 segundos).
- Cada cliente compra um número aleatório de bilhetes de cada vez (de 1 a 10 bilhetes);
- Quando não há mais bilhetes disponíveis, as threads (clientes) morrem, libertando convenientemente todos os recursos usados;
- Assuma-se que o número máximo de threads (clientes) que podem comprar bilhetes ao mesmo tempo se limita a MAX_RUN_THREADS (e.g., se N_CLIENTS = 10 e

MAX_RUN_THREADS = 4, significa que: dos 10 clientes existentes, apenas 4 poderão fazer pedidos de compra de bilhetes ao mesmo tempo, porque a casa tem apenas 4 vendedores. Os restantes clientes esperaram a sua oportunidade para comprar);

- Depois de todas as threads clientes morrerem (em princípio porque os bilhetes esgotaram), a thread principal imprime no ecrã o número total de bilhetes vendidos.

Pense numa possível solução para o problema apresentado. Tenha em conta quais as variáveis partilhadas que poderão ser necessárias, bem como os recursos de sincronização, e qual o fluxo que terá a thread que representa o cliente.

Complete o código base fornecido de maneira a respeitar os requisitos solicitados.

Exercício 2. Impressão Sincronizada

Pretende-se sincronizar 3 tarefas, cada uma delas responsável por imprimir no ecrã uma das palavras da frase “Portugal Somos Nos!”. A frase completa deve ser escrita 100 vezes, uma por linha (i.e. tarefa1 imprime 100x “Portugal”; tarefa2 imprime 100x “Somos”; tarefa3 imprime 100x “Nos!”). O output do programa deve ser semelhante ao seguinte:

```
Portugal Somos Nos!  
Portugal Somos Nos!  
Portugal Somos Nos!  
Portugal Somos Nos!  
Portugal Somos Nos!  
Portugal Somos Nos!  
Portugal Somos Nos!  
Portugal Somos Nos!  
Portugal Somos Nos!  
Exiting main...
```

- a) Desenvolva um programa que implemente o descrito recorrendo ao uso de semáforos.
- b) Em vez de utilizar tarefas, experimente fazer o mesmo com a utilização de processos e recorrendo a ‘named semaphores’ [Ex.2b: pode auxiliar a resolução com recurso a ferramentas de IA].